Better GBFV Bootstrapping and Faster Encrypted Edit Distance Computation

Robin Geelen and Frederik Vercauteren

COSIC, KU Leuven, Leuven, Belgium

Abstract. We propose a new iterative method to convert a ciphertext from the Generalized BFV (GBFV) to the regular BFV scheme. In particular, our conversion starts from an encrypted plaintext that lives in a large cyclotomic ring modulo a small-norm polynomial $t(x)$, and gradually changes the encoding to a smaller cyclotomic ring modulo a much larger integer p. Previously, only a trivial conversion method was known, which did not change the underlying cyclotomic ring.

Using our improved conversion algorithm, we can bootstrap the GBFV scheme almost natively, in the sense that only a very small fraction of the operations is computed inside regular BFV. Specifically, we evaluate (an adapted version of) the slot-to-coefficient transformation entirely in the GBFV scheme, whereas the previous best method used the BFV scheme for that transformation. This insight allows us to bootstrap either with less noise growth, or much faster than the state-of-the-art.

We implement our new bootstrapping in Microsoft SEAL. Our experiments show that, for the same remaining noise budget, our bootstrapping runs in only 800 ms when working with ciphertexts containing 1024 slots over $\mathbb{F}_p$ with $p = 2^{16} + 1$. This is 1.6× faster than the state-of-the-art.

Finally, we use our improved GBFV bootstrapping in an application that computes an encrypted edit distance. Compared to the recent TFHE-based Leuvenshtein algorithm, our GBFV version is almost two orders of magnitude faster in the amortized sense.

Keywords: Fully homomorphic encryption · GBFV · Bootstrapping · Edit distance

1 Introduction

Applications of fully homomorphic encryption (FHE) often rely on parallel data processing. To accommodate this parallelism, many schemes (such as BGV [BGV14], BFV [Bra12, FV12] and CKKS [CKKS17]) provide a built-in mechanism for "single instruction, multiple data" (SIMD) operations. In typical settings, these schemes can pack thousands of data elements in a SIMD vector. Basic arithmetic operations are then evaluated entry-wise.

While the above SIMD schemes can pack a massive amount of data, certain applications only need a very moderate packing capacity. Of course, it is possible to use a subset of the available SIMD slots, but this is quite wasteful because it leaves a large portion of the message space unused. Recently, a Generalized BFV (GBFV) scheme was proposed by Geelen and Vercauteren [GV25] and concurrently by Cha et al. [CHM+25]. Loosely speaking, the GBFV scheme decreases the size of the SIMD vector by some small integer factor α, resulting in a scheme that can be up to α times as efficient. This means that either the latency can be reduced, or the computational capacity increased. In other words, the GBFV scheme trades packing for efficiency. For the parameter sets considered in this paper, the main advantage of GBFV over regular BFV is its reduced latency.

Similarly to other FHE schemes, GBFV has a bootstrapping procedure to evaluate circuits of arbitrary size. The current bootstrapping algorithm can be understood as a

E-mail: robin.geelen@esat.kuleuven.be (corresponding) and frederik.vercauteren@esat.kuleuven.be

hybrid solution: its last step (called *digit removal*) is evaluated using GBFV directly; its first step (called *noisy expansion*) is instantiated in the BFV domain; connecting both steps is achieved via a conversion routine between GBFV and regular BFV. A major drawback of this hybrid approach is that it hurts the performance: the BFV scheme uses much more computational budget than GBFV, so the cost of bootstrapping becomes completely dominated by the noisy expansion step. Therefore, further improvements of this step are required to reduce the computational cost.

1.1 Contributions

The main observation of this paper is that noisy expansion does not use the available plaintext space efficiently. Since this step is evaluated in the BFV domain, only a $1/\alpha$-fraction of the slots contain relevant information and the others contain garbage. One solution to this problem is batch bootstrapping [GV25], but this is less interesting from a software engineering point of view, because multiple ciphertexts need to be collected (which may not even be available depending on the application) and then bootstrapped simultaneously. Therefore, we propose a bootstrapping algorithm for individual ciphertexts that makes better use of the available space by staying much longer in the GBFV domain. We contribute to the state-of-the-art as follows:

- We propose an improved noisy expansion step in which *almost all* operations are evaluated inside GBFV. As a result, we either retain more computational budget, or can bootstrap faster than the previous work. The core of our algorithm is a new subroutine that homomorphically converts between two isomorphic rings $\mathcal{R}'_t$ and $\mathcal{R}''_p$ with different encoding, and where $\mathcal{R}''$ is a subring of $\mathcal{R}'$. This is an iterative procedure consisting of $\min(\log_2(n''), \log_2(n'/n''))$ steps, where n' and n'' are the vector dimensions of $\mathcal{R}'$ and $\mathcal{R}''$ respectively. The previous best GBFV bootstrapping algorithm used a trivial conversion from $\mathcal{R}'_t$ to $\mathcal{R}'_p$, instead of $\mathcal{R}''_p$. Our implementation in Microsoft SEAL shows that we can bootstrap a ciphertext of 4096 slots in only a second, while still having 6 remaining multiplicative levels.[1]
- Using our improved bootstrapping algorithm, we present a GBFV-tailored algorithm to homomorphically compute the Levenshtein distance of two encrypted strings. The Levenshtein distance computes the minimal number of deletions, insertions and substitutions that turns the first string into the second. Measuring amortized performance, our GBFV version is approximately two orders of magnitude faster than the state-of-the-art solution, called Leuvenshtein [LDS+25], which takes a TFHE-based approach.

2 Preliminaries

This section recaps the relevant aspects of GBFV – a scheme that was proposed independently by Geelen and Vercauteren [GV25] and by Cha et al. [CHM+25]. We restrict our bootstrapping to power-of-two cyclotomics. Consider the $2n$-th cyclotomic ring $\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$ and its subring $\mathcal{R}' = \mathbb{Z}[y]/(y^{n'} + 1)$ (it can be seen that this is indeed a subring via the homomorphism $y \mapsto x^{n/n'}$). We can apply ring automorphisms $\sigma_i \colon x \mapsto x^i$ to elements of $\mathcal{R}$ for any odd integer $i \in \mathbb{Z}_{2n}^\times$. It is well known that the automorphism group has rank two and can be decomposed as $\langle \sigma_{-1} \rangle \times \langle \sigma_5 \rangle$. Consider a modulus $t = t(x) \in \mathcal{R}$ (not necessarily a scalar) and define the quotient $\mathcal{R}_t = \mathcal{R}/t\mathcal{R}$. Elements of $\mathcal{R}$ or $\mathcal{R}_t$ are written as $\boldsymbol{m} = m(x)$ and will be plaintexts of our encryption scheme.

The GBFV scheme generalizes BFV [Bra12, FV12] and CLPX [CLPX18]: the modulus t is a scalar value in BFV, a linear polynomial in CLPX and an arbitrary non-zero element

[1] See https://github.com/KULeuven-COSIC/Bootstrapping_BGV_BFV/tree/traces.

in GBFV. We omit the definitions of key generation, encryption and decryption. GBFV allows the following homomorphic operations:

- $\mathsf{GBFV.Add}(\mathsf{ct}_1, \mathsf{ct}_2) \to \mathsf{ct}_{\mathsf{add}}$: given ct_1 and ct_2 that encrypt $\boldsymbol{m}_1, \boldsymbol{m}_2 \in \mathcal{R}_t$ respectively, compute $\mathsf{ct}_{\mathsf{add}}$ that encrypts $\boldsymbol{m}_{\mathsf{add}} = \boldsymbol{m}_1 + \boldsymbol{m}_2$.
- $\mathsf{GBFV.Sub}(\mathsf{ct}_1, \mathsf{ct}_2) \to \mathsf{ct}_{\mathsf{sub}}$: given ct_1 and ct_2 that encrypt $\boldsymbol{m}_1, \boldsymbol{m}_2 \in \mathcal{R}_t$ respectively, compute $\mathsf{ct}_{\mathsf{sub}}$ that encrypts $\boldsymbol{m}_{\mathsf{sub}} = \boldsymbol{m}_1 - \boldsymbol{m}_2$.
- $\mathsf{GBFV.Mul}(\mathsf{ct}_1, \mathsf{ct}_2) \to \mathsf{ct}_{\mathsf{mul}}$: given ct_1 and ct_2 that encrypt $\boldsymbol{m}_1, \boldsymbol{m}_2 \in \mathcal{R}_t$ respectively, compute $\mathsf{ct}_{\mathsf{mul}}$ that encrypts $\boldsymbol{m}_{\mathsf{mul}} = \boldsymbol{m}_1 \cdot \boldsymbol{m}_2$.
- $\mathsf{GBFV.Auto}(\mathsf{ct}, \sigma_i) \to \mathsf{ct}_{\mathsf{auto}}$: given ct that encrypts $\boldsymbol{m} \in \mathcal{R}_t$ and σ_i such that $\sigma_i(t) = t$, compute $\mathsf{ct}_{\mathsf{auto}}$ that encrypts $\boldsymbol{m}_{\mathsf{auto}} = \sigma_i(\boldsymbol{m})$.[2]
- $\mathsf{GBFV.ModUp}(\mathsf{ct}, t_2) \to \mathsf{ct}_{\mathsf{up}}$: given ct that encrypts $\boldsymbol{m} \in \mathcal{R}_{t_1}$, and $t_2 \in t_1\mathcal{R}$, compute $\mathsf{ct}_{\mathsf{up}}$ that encrypts $\boldsymbol{m}_{\mathsf{up}} \in \mathcal{R}_{t_2}$ such that $\boldsymbol{m}_{\mathsf{up}} = \boldsymbol{m} \pmod{t_1\mathcal{R}}$, and additionally we have $\boldsymbol{m}_{\mathsf{up}} = 0 \pmod{(t_2/t_1)\mathcal{R}}$.
- $\mathsf{GBFV.ModDown}(\mathsf{ct}, t_2) \to \mathsf{ct}_{\mathsf{down}}$: given ct that encrypts $\boldsymbol{m} \in \mathcal{R}_{t_1}$, and t_2 such that $t_1 \in t_2\mathcal{R}$, compute $\mathsf{ct}_{\mathsf{down}}$ that encrypts $\boldsymbol{m}_{\mathsf{down}} = \boldsymbol{m} \pmod{t_2\mathcal{R}}$ in the ring $\mathcal{R}_{t_2}$.
- $\mathsf{GBFV.Div}(\mathsf{ct}, d) \to \mathsf{ct}_{\mathsf{div}}$: given ct that encrypts $d \cdot \boldsymbol{m} \in \mathcal{R}_t$, and d such that $t \in d\mathcal{R}$, compute $\mathsf{ct}_{\mathsf{div}}$ that encrypts $\boldsymbol{m} \in \mathcal{R}_{t/d}$.
- $\mathsf{GBFV.InProd}(\mathsf{ct}) \to \mathsf{ct}_{\mathsf{fresh}}$: given ct that encrypts $\boldsymbol{m} \in \mathcal{R}_p$ for an integer p, compute $\mathsf{ct}_{\mathsf{fresh}}$ that encrypts $\boldsymbol{m}_{\mathsf{fresh}} = p \cdot \boldsymbol{m} + \boldsymbol{e} \in \mathcal{R}_{p^2}$, where $\boldsymbol{e}$ is an error polynomial with small coefficients.

Observe that BFV-to-GBFV conversion from Geelen and Vercauteren [GV25] is a special case of ModDown, while ModUp is a stronger notion than GBFV-to-BFV conversion (since we additionally impose that $\boldsymbol{m}_{\mathsf{up}} = 0 \pmod{(t_2/t_1)\mathcal{R}}$). These routines can be implemented as follows: ModUp corresponds to a multiplication by $(t_2/t_1)^{-1} \pmod{t_1}$, and ModDown is a multiplication by t_1/t_2.

2.1 Encoding in Plaintext Slots

Consider any power-of-two cyclotomic ring $\mathcal{R}$ (we will fix particular rings later on). The plaintext space of GBFV is

$$\mathcal{R}_t = \mathbb{Z}[x]/(x^n + 1, t(x)).$$

Let $p \in \mathbb{Z}$ be the characteristic of $\mathcal{R}_t$ (i.e. the smallest positive integer in $t\mathcal{R}$), which we assume to be a prime number congruent to 1 modulo $2n$. This restriction gives us base field encoding (i.e. the slots live in $\mathbb{F}_p$) and is most commonly used in (G)BFV to maximize the SIMD parallelism. We now need a mechanism for writing plaintext slots as a vector. Since we have a prime characteristic, we can work in the principal ideal domain $\mathbb{F}_p[x]$. It follows that

$$(x^n + 1, t(x)) = (\tau(x), p) \tag{1}$$

as ideals in $\mathbb{Z}[x]$, where $\tau(x) = \gcd(x^n + 1, t(x))$ is computed in $\mathbb{F}_p[x]$.

A root of $\tau(x)$ over $\mathbb{F}_p$ is also a root of $x^n + 1$, and hence a primitive $2n$-th root of unity. Let $\zeta \in \mathbb{F}_p$ be such a root, then the other ones are of the form

$$y = \zeta^{(-1)^\alpha \cdot 5^\beta}, \tag{2}$$

though not necessarily each element of this shape is a root of $\tau(x)$. We enforce an order on these roots and the corresponding plaintext slots they define. More precisely, let S be the set of roots of $\tau(x)$, with order relation $\leq$ such that

$$\zeta^{(-1)^\alpha \cdot 5^\beta} \leq \zeta^{(-1)^\gamma \cdot 5^\delta}$$

[2] A possible relaxation is that σ_i stabilizes $t\mathcal{R}$ rather than t, but both conditions are equivalent for our parameter sets.

7	15
6	14
5	13
4	12
3	11
2	10
1	9
0	8

(a) Conventional ordering

14	15
12	13
10	11
8	9
6	7
4	5
2	3
0	1

(b) Our ordering

3	
2	
1	
0	

(c) Conventional/our ordering

Figure 1: Examples of conventional slot ordering and our slot ordering

if either $\beta < \delta$, or $\beta = \delta$ and $\alpha \leq \gamma$. This definition assumes that $0 \leq \alpha, \gamma < 2$ and $0 \leq \beta, \delta < n/2$. The plaintext slot-vector is defined using the isomorphism

$$\begin{aligned} \mathcal{R}_t = \mathbb{Z}[x]/(\tau(x), p) &\to \mathbb{F}_p^{|S|} \\ m(x) &\mapsto \{m(s)\}_{s \in S}\,. \end{aligned}$$

Addition and multiplication act component-wise on the vector of plaintext slots, and rotations can be implemented with automorphisms. The slot encoding can be generalized to any prime-power characteristic [GV25], simply by substituting $\mathcal{R}_t$ with $\mathcal{R}_{t^e}$, which results in characteristic p^e (in practice, we use $e = 1$ or $e = 2$).

Note that our slot ordering is quite non-standard: the more common definition in the BFV literature [Gee24] switches the role of α and β. Figure 1 clarifies the difference by comparing the conventional ordering to ours. This figure visualizes the array of slots, where the horizontal and vertical dimensions represent generators -1 and 5 respectively. The first and second subfigures correspond to the regular BFV scheme, where all slots are filled. In the GBFV scheme, only a subset of the slots is used (the unused slots are drawn as empty boxes in the third subfigure). We believe that our ordering is more natural due to the following reasons:

- Noisy expansion maps slots to coefficients in a permuted order. In the standard definition, this permutation is a bit reversal in both hypercolumns. Our ordering results in one full bit reversal over the entire slot-vector, similarly to the plaintext number-theoretic transform (NTT) algorithm.
- This ordering facilitates the exposition of our subring conversion algorithm. When increasing the plaintext modulus, additional slots with odd-numbered indices will appear. Similarly, doubling the dimension of the cyclotomic ring will simply duplicate the slot-vector. These operations are more difficult to describe in the conventional slot ordering.
- Our conversion algorithm applies an additional permutation to the plaintext slots. In our ordering, this is a simple circular bit shift to the left.

2.2 Linear Transformations

The core of our new bootstrapping is an algorithm that homomorphically evaluates a particular $\mathbb{F}_p$-linear transformation on the vector of plaintext slots. This linear map converts between coefficient and slot representation. Below we discuss the state-of-the-art for implementing such linear transformations.

2.2.1 Baby-Step/Giant-Step Method

We implement "sparse" linear transformations with the baby-step/giant-step algorithm. In particular, we will use the generalized variant of Cheon et al. [CCLS19]. This method computes linear combinations of rotated ciphertexts by splitting the relevant index set into two components. More specifically, let $I = J \cdot K \subseteq \mathbb{Z}_{2n}^{\times}$. Then we can rewrite a linear map L over index set I as

$$L(\boldsymbol{m}) = \sum_{i \in I} \boldsymbol{\kappa}_i \cdot \sigma_i(\boldsymbol{m}) = \sum_{k \in K} \sigma_k \left[\sum_{j \in J} \boldsymbol{\kappa}'_{jk} \cdot \sigma_j(\boldsymbol{m}) \right],$$

where $\boldsymbol{\kappa}'_{jk} = \sigma_k^{-1}(\boldsymbol{\kappa}_{jk})$. The minimum number of automorphisms is reached when J and K have (approximately) the same cardinality. Our implementation uses a simple heuristic algorithm to determine J and K from I.

2.2.2 Trace Function

To take an encrypted plaintext from $\mathcal{R}$ to a subring $\mathcal{R}'$, we can homomorphically apply the trace function [AP13], which is defined as

$$\mathrm{Tr}_{\mathcal{R}/\mathcal{R}'} \colon \sum_{i=0}^{n-1} m_i \cdot x^i \mapsto (n/n') \cdot \left(\sum_{i=0}^{n'-1} m_{i \cdot n/n'} \cdot y^i \right). \tag{3}$$

It is well known that the trace can be computed iteratively, by passing through all intermediate cyclotomic subrings, and the cost is dominated by $\log_2(n/n')$ automorphisms. For ease of implementation, we use the trace method from Chen and Han [CH18], while some advanced methods have slightly less noise growth [CDKS21, WHS+25, LY25].

3 Bootstrapping

For the rest of this paper, we fix three particular cyclotomic rings: the *R-LWE ring* $\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$, which is used for ring learning with errors encryption (and hence determines the security level); the *GBFV ring* $\mathcal{R}' = \mathbb{Z}[y]/(y^{n'} + 1)$, which encodes GBFV plaintexts; and the *BFV ring* $\mathcal{R}'' = \mathbb{Z}[z]/(z^{n''} + 1)$, which encodes BFV plaintexts. Note that $n \geq n' \geq n''$ and hence $\mathcal{R}'' \subseteq \mathcal{R}' \subseteq \mathcal{R}$. We fix an initial plaintext modulus $t \in \mathcal{R}$ and let p be the smallest positive integer in $t\mathcal{R}$. The characteristic p stays the same during our subring conversion, but the plaintext modulus itself will gradually change from t to p.

We impose two more parameter restrictions: (1) we assume that $t(x)$ lives in the $(2n'/n'')$-th cyclotomic ring; and (2) let $\tau(x)$ be as defined in Equation (1), then $\tau(x)$ is assumed to have n'' roots in $\mathbb{F}_p$. Moreover, these roots should be generated by setting $\alpha = 0$ in Equation (2). It follows from these assumptions that $\mathcal{R}'_t$ and $\mathcal{R}''_p$ are isomorphic to $\mathbb{F}_p^{n''}$ as rings. We emphasize that these restrictions are very mild as they still support binomial plaintext moduli [GV25, CHM+25], which is the most common setting in practice. They also cover the parameter sets obtained by "descending" via the relative field norm, including the Goldilocks prime in power-of-two cyclotomics [GV25]. The only conceptual limitation is that the slot-vector of the input ciphertext needs to be one-dimensional as in Figure 1c. As such, we believe that this setting will not complicate parameter search.

3.1 Informal Overview

The crucial subroutine of bootstrapping that will be studied is noisy expansion. Informally, it homomorphically maps the SIMD vector as follows:

$$\mathbb{F}_p^{\ell} \to \mathbb{Z}_{p^2}^{\ell} \colon (m_0, \ldots, m_{\ell-1}) \mapsto (p \cdot m_0 + e_0, \ldots, p \cdot m_{\ell-1} + e_{\ell-1}),$$

where ℓ denotes the length of the SIMD vector. The plaintext space is expanded from $\mathbb{F}_p$ to $\mathbb{Z}_{p^2}$ and contains additional "noise" terms e_i. The output ciphertext should also contain significantly less noise than the input (i.e. it is refreshed).

The old method [GV25] implements noisy expansion as a trivial composition of three building blocks: SlotToCoeff, InProd and CoeffToSlot. The first transformation converts a slot-encoded ciphertext into a coefficient-encoded ciphertext, the middle transformation was already described previously and implements the actual refresh, and the third transformation is the inverse of the first but computed modulo p^2. We will focus on the SlotToCoeff step, which turns an encryption of the SIMD vector $(m_0, \ldots, m_{\ell-1})$ into a ciphertext encrypting the polynomial

$$m(x) = \sum_{i=0}^{\ell-1} m_{\mathsf{rev}_\ell(i)} \cdot y^i \in \mathcal{R}'_p, \tag{4}$$

where rev_ℓ is the standard bit-reversal permutation of $\log_2(\ell)$-bit elements. The old method computes SlotToCoeff and CoeffToSlot entirely under BFV encoding, where the number of slots is $\ell = n'$. These are linear transformations that can be decomposed into a series of up to $\log_2(\ell)$ sparse matrices using a homomorphic NTT algorithm [Gee24, MHWW24b].

A first idea to improve the SlotToCoeff transformation could be to evaluate its first couple of stages directly in the GBFV domain. Unfortunately, this requires the automorphism σ_{-1}, which does not fix the modulus t. Instead, our idea is to rely on a subring encoding. The crucial observation is that we only need to take care of $\ell = n''$ GBFV slots, rather than all $\ell = n'$ BFV slots from the previous method. These slots can be mapped to the coefficients of a BFV plaintext in $\mathcal{R}''$. Our method consists of two components, which we describe in opposite order:

- The second component is an algorithm that converts a GBFV ciphertext to a BFV ciphertext that lives in the smallest possible cyclotomic subring. This is basically a conversion from $\mathcal{R}'$ to $\mathcal{R}''$ that also changes the slot-encoding. One side effect of the algorithm is that it applies an additional permutation on the plaintext slots, which can be understood as a circular bit shift.
- The first component is evaluating a negacyclic NTT, which is similar to the original transformation but defined in a smaller dimension (in dimension n'' rather than n'). To compensate for the circular bit shift, this transformation is applied to a slot-vector permuted with the inverse circular bit shift (this does not increase the complexity of the algorithm).

The computational cost is roughly the same as the state-of-the-art SlotToCoeff transformation (the only additional cost is a cheap trace operation), but the new algorithm consumes substantially less computational budget. This is because the negacyclic NTT can be fully evaluated in the GBFV domain. We do not achieve arbitrary-precision GBFV bootstrapping, but in contrast to the previous work, only a very small fraction of the algorithm is evaluated with BFV. Hence the supported precision can be increased significantly.

We can exploit the reduced noise accumulation of our algorithm in three ways: either we use the same decomposition parameters for the SlotToCoeff transformation as the previous work, resulting in a ciphertext with more remaining computational budget. Alternatively, the GBFV scheme can be instantiated with a smaller ciphertext modulus, resulting in a faster execution time. Finally, we remark that the previous work decomposed the transformation in only two stages (which is quite limited in practice) making it the bottleneck in terms of execution time. With our improved method, each stage consumes less computational budget, so we can afford to decompose the transformation in more stages. This third strategy is faster and uses fewer evaluation keys, but it slightly increases the accumulated noise.

3.2 GBFV to Subring BFV Conversion

This section describes our new homomorphic conversion routine from the GBFV plaintext ring $\mathcal{R}'_t$ to the BFV plaintext ring $\mathcal{R}''_p$. Although both plaintext rings are isomorphic, their encoding is different, so that we cannot simply reinterpret a plaintext from one ring as a plaintext of the other one. Instead, our algorithm evaluates a particular $\mathbb{F}_p$-linear transformation on the plaintext. This is achieved by iteratively passing through a series of intermediate rings, where each step "recodes" the underlying plaintext. More specifically, we use the rings

$$\mathcal{R}' = \mathcal{R}^0 \supset \mathcal{R}^1 \supset \cdots \supset \mathcal{R}^L \supseteq \mathcal{R}'',$$

where $\mathcal{R}^i$ are consecutive power-of-two cyclotomic rings. Since each step goes to a smaller cyclotomic ring, we need to compensate by increasing the underlying plaintext modulus. Therefore, we consider the chain of moduli

$$t = t_0 \mid t_1 \mid \cdots \mid t_L \mid p,$$

where divisibility is defined in $\mathcal{R}$. We construct t_i by multiplying t_{i-1} with a shifted version of itself, in such a way that each $\mathcal{R}^i_{t_i}$ is isomorphic to $\mathbb{F}_p^\ell$ (where we fix $\ell = n''$ from now on). More details follow in the next paragraphs.

3.2.1 Homomorphic Subring Conversion

Our first task is converting a GBFV-encoded ciphertext to a BFV-encoded ciphertext. This will come with a side effect: an additional circular bit shift permutation $\mathsf{shift}_{\ell,L}$ will be applied to the order of the entries in the SIMD vector. This permutation shifts a $\log_2(\ell)$-bit number with L positions to the left, and it will come on top of the bit-reversal permutation from Equation (4). More specifically, if $\ell \geq n'/n''$, then subring conversion maps the slot with position j to the slot with position $\mathsf{shift}_{\ell,L}(j)$. If $\ell \leq n'/n''$, no overall circular bit shift is applied. This is because the number of iterations is $L = \log_2(\ell)$, which results in the identity permutation.

Many Plaintext Slots. If $\ell \geq n'/n''$, we proceed as follows. Let there be $L = \log_2(n'/n'')$ iterations in our algorithm and consider

$$g_i = 5^{-2^{L-2}}, \ldots, 5^{-2^1}, 5^{-2^0}, -1 \quad \text{for} \quad i = 1, \ldots, L. \tag{5}$$

Then we recursively define the next plaintext moduli as $t_i = t_{i-1} \cdot \sigma_{g_i}(t_{i-1})$ for each i.[3] Iteration i of our subring conversion does the following:

- Compute ModUp to t_i, which gives an encrypted plaintext $\boldsymbol{\mu}_i \in \mathcal{R}^{i-1}_{t_i}$. This plaintext has 2ℓ slots in total, the odd ones of which are identically zero.
- Next, we need to homomorphically map this plaintext to $\boldsymbol{m}_i \in \mathcal{R}^i_{t_i}$, which extracts the relevant ℓ slots. To this end, we define the "mask" $\boldsymbol{a}_i \in \mathcal{R}^{i-1}_{t_i}$ that encodes '1' in the first half and '0' in the second half of the slot-vector. Then we homomorphically evaluate the function

$$\mathsf{GBFV.SubPermute}_i \colon \boldsymbol{\mu}_i \mapsto \boldsymbol{a}_i \cdot \boldsymbol{\mu}_i + \sigma_{g_i}((1 - \boldsymbol{a}_i) \cdot \boldsymbol{\mu}_i).$$

- Finally, we compute $\boldsymbol{m}_i$ by taking the trace of $\mathcal{R}^{i-1}/\mathcal{R}^i$.

These steps are summarized in Algorithm 1 and visualized in Figure 2. The gray boxes are slots that are currently in use. Note that ModUp is merely a change of perspective: it can be thought of as creating extra room (i.e. gray boxes) for the permutation that

[3] This operation corresponds to computing the relative field norm of t_{i-1}.

follows. Each step maps to a smaller cyclotomic ring, which can be thought of as a subring of the bigger one having periodic slot structure. Since all information is captured within one period, we do not show repetitions in the figure. Apart from mapping to a subring, each iteration also applies the permutation $\mathsf{shift}_{\ell,1}$ to the order of the entries in the SIMD slot-vector, which eventually accumulates to $\mathsf{shift}_{\ell,L}$. This can be seen as follows:

- ModUp doubles the size of the slot-vector, and as a result, also doubles the index of each plaintext slot.
- SubPermute maps all slots in the second half from index j to index $j+1$.
- The trace divides all slots in pairs and maps the slots in the second half from index j to index $j-\ell$.

These steps result in

$$\mathsf{shift}_{\ell,1}(j) = \begin{cases} 2j & \text{if } j < \ell/2 \\ 2j - \ell + 1 & \text{if } j \geq \ell/2, \end{cases}$$

which is indeed a circular bit shift to the left with one position.

Few Plaintext Slots. If $\ell \leq n'/n''$, then the above method can be optimized by skipping some iterations. Intuitively, this works because the number of GBFV slots is relatively small, so we can map them to the BFV ring without passing through all intermediate cyclotomic rings. The only changes are the following:

- The number of iterations is $L = \log_2(\ell)$. The other steps, including computation of the automorphism indices from Equation (5), are unchanged.
- Since $\mathcal{R}^L \neq \mathcal{R}''$ and $t_L \neq p$, we post-process by converting the modulus to p and taking the trace to $\mathcal{R}''$.

These steps are summarized in Algorithm 1 and visualized in Figure 3. Note that no overall circular bit shift is applied since $\mathsf{shift}_{\ell,L}$ is the identity permutation. Notably, this version of our conversion algorithm violates the condition for an automorphism to be valid (since $\sigma_{g_i}(t_i) \neq t_i$), but Figure 3 shows that the result is still well-defined.[4] In the implementation, it suffices to compute the invalid automorphism temporarily in the BFV domain. Interestingly, the conversion algorithm has the largest number of iterations (and is hence least efficient) if $\ell = n'/n''$, which is when the versions for many and few plaintext slots coincide.

Algorithm 1 Homomorphic subring conversion

Require: ct_0 that encrypts $\boldsymbol{m}_0 \in \mathcal{R}'_t$
Ensure: ct_L that encrypts $\boldsymbol{m}_L \in \mathcal{R}''_p$, encoding the same slots as $\boldsymbol{m}_0$

function GBFV.SubConvert(ct_0)
  $L \leftarrow \min(\log_2(\ell), \log_2(n'/n''))$ ▷ Number of iterations
  for $i \leftarrow 1$ to L **do**
    $\mathsf{ct}_i \leftarrow \mathsf{ModUp}(\mathsf{ct}_{i-1}, t_i)$
    $\mathsf{ct}_i \leftarrow \mathsf{SubPermute}_i(\mathsf{ct}_i)$
    $\mathsf{ct}_i \leftarrow \mathrm{Tr}_{\mathcal{R}^{i-1}/\mathcal{R}^i}(\mathsf{ct}_i)$
  end for
  $\mathsf{ct}_L \leftarrow \mathrm{Tr}_{\mathcal{R}^L/\mathcal{R}''}(\mathsf{ModUp}(\mathsf{ct}_L, p))$ ▷ Void if $\ell \geq n'/n''$
  return ct_L
end function

[4] The automorphism is well-defined by enforcing its output to be a multiple of t_{i-1}.

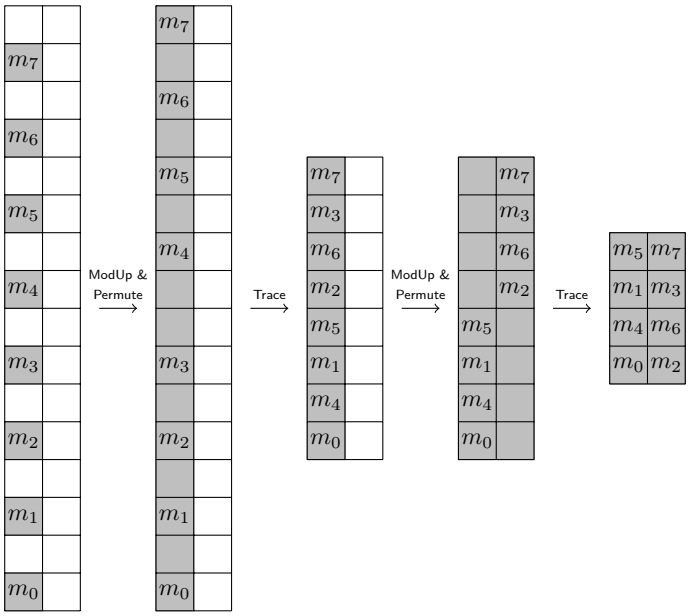

Figure 2: Subring conversion for many plaintext slots (with $n' = 32$ and $n'' = 8$)

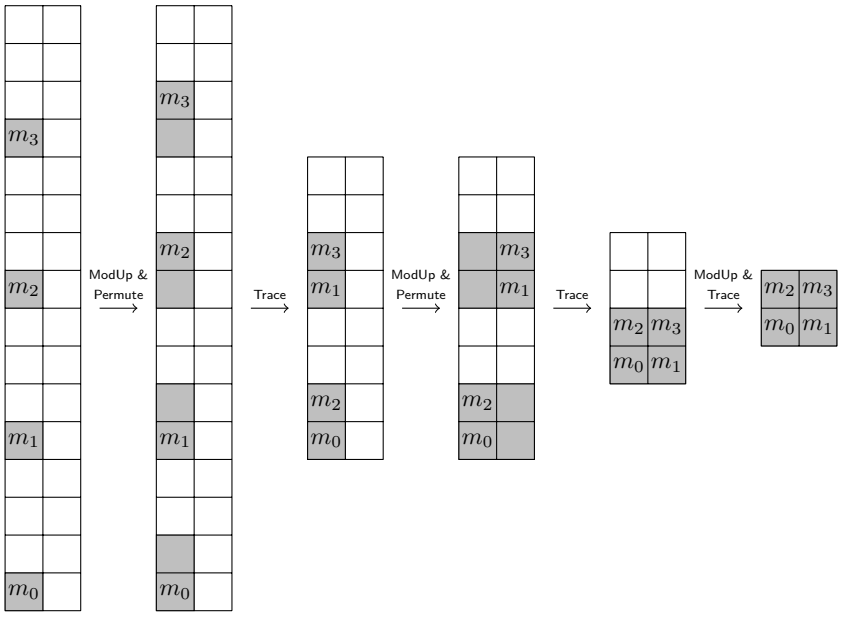

Figure 3: Subring conversion for few plaintext slots (with $n' = 32$ and $n'' = 4$)

Optimizations. Algorithm 1 has multiplicative depth L, which can be reduced by merging multiple iterations (a strategy reminiscent of level-collapsing in homomorphic FFT evaluation [CCS19]). To enable this optimization, we can implement $\mathsf{SubPermute}$ as a larger linear transformation with the baby-step/giant-step algorithm. Another improvement is merging the level of ModUp into $\mathsf{SubPermute}$. As explained previously, we implement ModUp as a constant multiplication, which can be folded in the subsequent or previous linear transformation by adapting the definition of $\boldsymbol{a}_i$. The only exception is when we encrypt a single slot, because there is no linear transformation in that case. In the bootstrapping application, the constant can still be folded into the adapted digit removal polynomial.

3.2.2 Homomorphic Extension Ring Conversion

Our bootstrapping algorithm also requires the inverse operation of subring conversion: converting back to the original extension ring. Algorithm 2 specifies our extension ring conversion. A notable difference is that extension ring conversion is defined in characteristic p^2 to accommodate for the extra noise terms present in the plaintext. Specifically, it maps an element from $\mathcal{R}''_{p^2}$ to the isomorphic ring $\mathcal{R}'_{t^2}$.

The basic idea of Algorithm 2 is to run all iterations of subring conversion in reverse order and under the inverse operations. Only the trace has no inverse and is replaced by reinterpreting an $\mathcal{R}^i$-encryption as an $\mathcal{R}^{i-1}$-encryption. The inverse of $\mathsf{SubPermute}_i$ is

$$\mathsf{GBFV.ExtPermute}_i \colon \boldsymbol{\mu}_i \mapsto \boldsymbol{a}_i \cdot \boldsymbol{\mu}_i + (1 - \boldsymbol{a}_i) \cdot \sigma_{g_i}^{-1}(\boldsymbol{\mu}_i),$$

where the same $\boldsymbol{a}_i$ as previously are now lifted to $\mathcal{R}_{t_i^2}^{i-1}$.

Algorithm 2 Homomorphic extension ring conversion

Require: ct_0 that encrypts $\boldsymbol{m}_0 \in \mathcal{R}''_{p^2}$
Ensure: ct_L that encrypts $\boldsymbol{m}_L \in \mathcal{R}'_{t^2}$, encoding the same slots as $\boldsymbol{m}_0$

```
function GBFV.ExtConvert(ct_0)
    L ← min(log_2(ℓ), log_2(n'/n''))          ▷ Number of iterations
    for i ← 1 to L do
        ct_i ← ExtPermute_{L−i+1}(ct_{i−1})
        ct_i ← ModDown(ct_i, t^2_{L−i})
    end for
    return ct_L
end function
```

3.3 Improved Noisy Expansion

Algorithm 3 specifies our improved noisy expansion. It has the following steps:

- Homomorphically multiply the SIMD vector by an adapted NTT matrix: let $U \in \mathbb{F}_p^{\ell \times \ell}$ be the usual NTT matrix [Gee24, MHWW24b], and consider the permutation matrix P that implements $\mathsf{shift}_{\ell,L}$. The adapted NTT matrix is defined as $V = P^{-1}UP$ (i.e. a simple change of basis), and we will homomorphically multiply the SIMD slot-vector by V. The same level-collapsing decomposition is applied as in the previous works [Gee24, MHWW24b] but in the basis determined by P: for example, if the usual decomposition is $U = U_2 \cdot U_1$, then we have $V = V_2 \cdot V_1$ with $V_i = P^{-1}U_iP$. This step can therefore be understood as evaluating the same transformation on a permuted slot-vector.
- Convert the resulting GBFV ciphertext to BFV with our new algorithm. A side effect of this step is multiplication by P, so the full transformation thus far is $PV = UP$.

- Refresh the ciphertext using InProd and take the trace to $\mathcal{R}''$, which removes the redundant coefficients.
- Convert the resulting ciphertext back to the extension ring.
- Homomorphically multiply the slot-vector by $((n/n'') \cdot V)^{-1}$, which is defined as a matrix in $\mathbb{Z}_{p^2}^{\ell \times \ell}$. This step also compensates for the additional factor of (n/n'') in the trace (cf. Equation (3)).

Similarly to how we apply level-collapsing within multiplication by V and within $\mathsf{SubConvert}$, we can also apply it across these building blocks. More details follow in Section 3.5.

Algorithm 3 Noisy expansion

Require: ct that encrypts $\boldsymbol{m} \in \mathcal{R}'_t$
Ensure: $\mathsf{ct}_{\mathsf{exp}}$ that encrypts $\boldsymbol{m}_{\mathsf{exp}} \in \mathcal{R}'_{t^2}$, encoding a noisy version of the slots of $\boldsymbol{m}$

function GBFV.NoisyExpand(ct)
  $\mathsf{ct}_{\mathsf{lt}} \leftarrow V \cdot \mathsf{ct}$ ▷ Adapted $\mathsf{SlotToCoeff}$
  $\mathsf{ct}_{\mathsf{sub}} \leftarrow \mathsf{SubConvert}(\mathsf{ct}_{\mathsf{lt}})$
  $\mathsf{ct}_{\mathsf{fresh}} \leftarrow \mathrm{Tr}_{\mathcal{R}/\mathcal{R}''}(\mathsf{InProd}(\mathsf{ct}_{\mathsf{sub}}))$
  $\mathsf{ct}_{\mathsf{ext}} \leftarrow \mathsf{ExtConvert}(\mathsf{ct}_{\mathsf{fresh}})$
  $\mathsf{ct}_{\mathsf{exp}} \leftarrow ((n/n'') \cdot V)^{-1} \cdot \mathsf{ct}_{\mathsf{ext}}$ ▷ Adapted $\mathsf{CoeffToSlot}$
  return $\mathsf{ct}_{\mathsf{exp}}$
end function

3.4 Our Bootstrapping Algorithm

We summarize our bootstrapping in Algorithm 4. First, it applies noisy expansion to the input ciphertext, and then it removes the additional noise terms via a slot-wise digit removal polynomial. Our bootstrapping uses the digit removal procedure from Ma et al. [MHWW24a], more specifically their algorithm over $\mathbb{Z}_{p^2}$. We refer to the appendix of [Gee24] for a simplified computation of $H(x)$. Finally, we need to divide the result by p, which is done in two steps: (1) multiplication by $(p/t)^{-1} \pmod t$; and (2) exact division by t, which will restore the plaintext modulus from t^2 to t. In an implementation, the first step can be folded in the coefficients of $H(x)$.

Algorithm 4 Bootstrapping

Require: ct that encrypts $\boldsymbol{m} \in \mathcal{R}'_t$
Ensure: $\mathsf{ct}_{\mathsf{boot}}$ that encrypts $\boldsymbol{m} \in \mathcal{R}'_t$ with less noise

function GBFV.Bootstrap(ct)
  $\mathsf{ct}_{\mathsf{exp}} \leftarrow \mathsf{NoisyExpand}(\mathsf{ct})$
  $\mathsf{ct}_{\mathsf{rem}} \leftarrow \mathsf{Mul}(H(\mathsf{ct}_{\mathsf{exp}}), (p/t)^{-1})$ ▷ Adapted digit removal
  $\mathsf{ct}_{\mathsf{boot}} \leftarrow \mathsf{Div}(\mathsf{ct}_{\mathsf{rem}}, t)$ ▷ Mere reinterpretation
  return $\mathsf{ct}_{\mathsf{boot}}$
end function

3.5 Complexity Analysis

Neglecting the field trace (which can be evaluated in logarithmic time and without level consumption), noisy expansion consists of an equally expensive forward and inverse transformation. As such, it suffices to study the forward transformation (i.e. multiplication with the adapted $\mathsf{SlotToCoeff}$ matrix V in conjunction with $\mathsf{SubConvert}$). In fully decomposed format, multiplication by V requires $\log_2(\ell)$ iterations [Gee24, MHWW24b]. Each iteration

is a linear transformation that can be implemented using 2 (or sometimes 1) automorphisms, and 3 (or sometimes 2) plaintext-ciphertext multiplications. Furthermore, our SubConvert algorithm has $L = \min(\log_2(\ell), \log_2(n'/n''))$ iterations, each of which needs 1 automorphism and 2 plaintext-ciphertext multiplications.

As mentioned previously, we do not need to fully decompose the above transformation, and we can instead apply level-collapsing in each phase. In short, we choose decomposition parameters $L_1, \ldots, L_s$ such that $L_1 + \ldots + L_s = L + \log_2(\ell)$. We then merge the first L_1 iterations in a first stage, the next L_2 iterations in a second stage, and so on. Each stage consumes one multiplicative level, and it requires $\mathcal{O}(2^{L_i/2})$ automorphisms and $\mathcal{O}(2^{L_i})$ plaintext-ciphertext multiplications using the baby-step/giant-step algorithm.

We believe that the most useful parameter sets in practice will be those with many slots (i.e. having $\ell \geq n'/n''$). Note that the total number of iterations in that case, including SlotToCoeff and SubConvert, is equal to $\log_2(n')$. This is the same as in the previous method [GV25], so the time complexity under the same decomposition parameters is identical. The improvement is in terms of noise growth: since the last stage typically covers the entire SubConvert algorithm, and SlotToCoeff is implemented in the other $s-1$ stages, the first $s-1$ stages can be fully evaluated with GBFV. Moreover, because of the reduced noise growth, we can afford to decompose the linear transformation into more stages than the previous work. This does slightly increase the noise growth again, but it reduces the execution time. While the previous work [GV25] decomposed the linear transformation in a rather limited number of $s = 2$ stages for ring dimension $n = 2^{14}$, we will also explore 3-stage and 4-stage decompositions.

4 Homomorphic Edit Distance

We now present our new algorithm to homomorphically compute the edit distance, more precisely the Levenshtein distance, leveraging our improved GBFV bootstrapping. The main reason to select this application is that it is very suitable to be implemented with GBFV: it combines a high multiplicative depth with a reasonable packing requirement. Moreover, the Levenshtein distance has several important privacy-preserving applications, including plagiarism detection [SAE+08] and DNA sequence alignment [ZLS+19]. Given two input strings $\mathbf{a}$ and $\mathbf{b}$, the Levenshtein distance $\Delta(\mathbf{a}, \mathbf{b})$ is the minimal number of operations needed to transform the first input string into the second. We consider three operations that all have unit cost: insertion, deletion and substitution.

4.1 Algorithmic Variants

4.1.1 Wagner-Fisher

As in previous works [CKL15, LDS+25] on homomorphic edit distance, the starting point is the Wagner-Fisher [WF74] algorithm, which serves as a basis for the actual algorithm. Given two input strings

$$\mathbf{a} = a_1 \ldots a_n \quad \text{and} \quad \mathbf{b} = b_1 \ldots b_m,$$

this algorithm computes an $(n+1) \times (m+1)$ distance matrix D. This matrix contains $D[i,j] = \Delta(\mathbf{a}[1:i], \mathbf{b}[1:j])$ for indices $0 \leq i \leq n$ and $0 \leq j \leq m$, where by definition $\mathbf{a}[1:0]$ and $\mathbf{b}[1:0]$ are empty strings. The first row thus satisfies $D[0,j] = j$, and similarly we have $D[i,0] = i$ for the first column. Since each operation is assumed to have unit cost, we can compute $D[i,j]$ for $i,j > 0$ as

$$D[i,j] = \begin{cases} D[i-1, j-1] & \text{if } a_i = b_j \\ 1 + \min(D[i-1,j], D[i,j-1], D[i-1,j-1]) & \text{otherwise.} \end{cases} \tag{6}$$

The value $D[n, m]$ will then give the actual Levenshtein distance. The definition of $D[i, j]$ already hints that an anti-diagonal approach is most natural, since the k-th anti-diagonal only depends on the $(k-1)$-th and $(k-2)$-th anti-diagonals.

To compute the above homomorphically, we thus need an efficient equality test and an efficient method to compute the minimum of three values. Note that the equality test is only computed on freshly encrypted ciphertexts, whereas the minimum function is not. Since the Levenshtein distance is limited to $n + m$, we know that the inputs to the minimum function are bounded by this value. However, computing such minimum is still very expensive homomorphically, which is why we switch to the Myers variant of the Wagner-Fischer algorithm.

4.1.2 Myers

The Myers [Mye99] algorithm is a differential version of Wagner-Fischer: it computes an $(n+1) \times m$ matrix H and an $n \times (m+1)$ matrix V containing the differences between horizontal and vertical values in D, so in particular

$$H[i, j] = D[i, j] - D[i, j-1] \quad \text{and} \quad V[i, j] = D[i, j] - D[i-1, j].$$

The first row of H and the first column of V contain all 1's. By substituting how the values of $D[i, j]$ are defined in Equation (6), we can compute the difference matrices H and V for $0 < i \leq n$ and $0 < j \leq m$ as

$$\begin{aligned} H[i, j] &= 1 - V[i, j-1] + \min(-\delta_{i,j}, V[i, j-1], H[i-1, j]) \\ V[i, j] &= 1 - H[i-1, j] + \min(-\delta_{i,j}, V[i, j-1], H[i-1, j]), \end{aligned}$$

where $\delta_{i,j} = 1$ if $a_i = b_j$ and $\delta_{i,j} = 0$ otherwise. The Levenshtein distance can then be recovered by computing the sum of the entries in H and V that lie on any path from index $(0, 0)$ to index (n, m), e.g. one could first compute the sum of the first row of H and then add the sum of the last column of V. Furthermore, since the first row consists of all 1's, the Levenshtein distance is recovered as

$$\Delta(\mathbf{a}, \mathbf{b}) = m + \sum_{i=1}^{n} V[i, m].$$

Although Myers computes two matrices instead of one in the Wagner-Fischer algorithm, the non-linear part (i.e. the minimum function) is common for both approaches. Furthermore, the main advantage of the above method compared to Wagner-Fischer is that we only need to evaluate the minimum function on the restricted domain $S = \{-1, 0\} \times \{-1, 0, 1\}^2$. As shown later, this function can be interpolated with a fairly low-degree polynomial.

Figure 4 shows an edit distance computation between the acronyms "FHE" and "HFE", similar to the visualization of Legiest et al. [LDS+25]. Our figure shows both the absolute distance matrix D (used in the Wagner-Fischer algorithm), and the horizontal and vertical difference matrices H and V (used in the Myers variant). The edit distance $\Delta(\text{FHE}, \text{HFE}) = 2$ is obtained in the bottom right. That is, one word can be transformed into the other via two substitutions, or via one insertion and one deletion.

4.1.3 Homomorphic Edit Distance Computation

Since we work in a finite field of odd characteristic, we can represent the minimum function $\min(x, y, z)$ on S by the following polynomial expression: let $h(t) = (1 - t) \cdot t$, then define

$$g(x, y, z) = x + ((1 + x)/4) \cdot (2h(y) + 2h(z) + h(y) \cdot h(z)) \quad \text{for } (x, y, z) \in S.$$

One can easily check that $g(x, y, z) = \min(x, y, z)$ on the domain S. Moreover, it is clear that g can be computed with a depth-3 circuit.

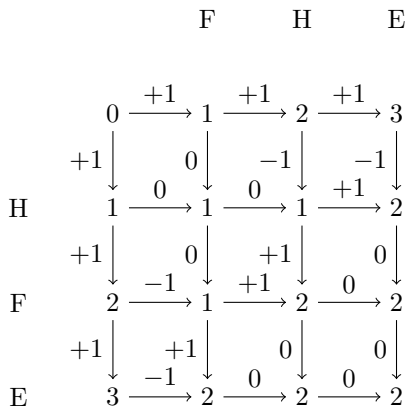

Figure 4: Example edit distance computation between FHE and HFE

Assuming the characters belong to some alphabet $\mathcal{A}$, we will simply encode them as the set $\{0, \ldots, |\mathcal{A}| - 1\}$. This requires that the plaintext space can represent this set. In our application, we use $p = 2^{16} + 1$, which allows us to even represent 2-byte encodings, e.g. UTF-16. The complexity of the equality test directly depends on the size of $\mathcal{A}$. That is, using our encoding, we can first compute $z_{i,j} = (a_i - b_j)^2$, which only has $|\mathcal{A}|$ possible values, namely $k^2 \bmod p$ for $k = 0, \ldots, |\mathcal{A}| - 1$. Let $e(x)$ denote the polynomial that maps all these values to 0, except 0 which is mapped to 1, then clearly $\delta_{i,j} = e(z_{i,j})$. The total degree is $2(|\mathcal{A}| - 1)$, so the depth of the equality test is $\lceil \log_2(|\mathcal{A}|) \rceil + 1$. The standard 7-bit ASCII encoding has depth 8. When using the full domain $\mathbb{F}_p$, the equality test becomes $1 - (a_i - b_j)^{p-1}$ due to Fermat's little theorem, resulting in depth 16 for the above prime.

Using the minimum and equality subroutines, we can formulate the homomorphic edit distance algorithm. For simplicity, we assume that both strings have length m and that our GBFV instance can encode up to ℓ slots. In practice, we take $p = 2^{16} + 1$ and $\ell = 4096$ for a ring dimension of $n = 2^{14}$.

The main use case of edit distance computation is that one encrypted input string $\mathbf{b} = b_1 \ldots b_m$ needs to be compared with a (typically large) number N of encrypted input strings $\mathbf{a}^{(i)}$. Since the GBFV instance can manipulate vectors of length ℓ, and the intermediate values in the computation consist of m values per edit distance, we will be able to process ℓ/m edit distance computations in parallel, where m is a power of two. By repeating this routine $\lceil Nm/\ell \rceil$ times, we can process N strings. To simplify notation, we will specify the algorithm for a single distance computation between $\mathbf{a}$ and $\mathbf{b}$. The parallel version is similar, where the only difference is that ℓ/m vectors of length m are packed in a single ciphertext. Specifically, our implementation uses an "interleaved" packing that merges the vectors, so that their circular rotations do not interfere.

From the definition of H and V, we see that (like the Wagner-Fischer algorithm), it is possible to compute them in an anti-diagonal manner. In the k-th step we will therefore compute the k-th anti-diagonal of the matrices H and V. These anti-diagonals are packed in ciphertexts ct_H (resp. ct_V), which in the k-th step (for $k \leq m$) of the algorithm contain the length-m vectors

$$[H[0,k], \ldots, H[k-1,1], 0, \ldots 0]$$
$$[V[1,k-1], \ldots, V[k,0], 0, \ldots 0].$$

For $k > m$, the anti-diagonals shrink in size, and the ciphertexts ct_H (resp. ct_V) contain

$$[H[k-m,m], \ldots, H[m,k-m], 0, \ldots, 0]$$
$$[V[k-m,m], \ldots, V[m,k-m], 0, \ldots, 0].$$

The actual algorithm does not explicitly put the last components of the above vector to zero (they may contain garbage values), but the first components will match. This saves unnecessary masking operations.

The only data that are still missing are the $\delta_{i,j}$. Since these are also needed in the same anti-diagonal order, and we want to compute them in the slots, we first duplicate the entries of $\mathbf{a}$ to obtain $M = \lceil m^2/\ell \rceil$ ciphertexts (or up to $M = m$ ciphertexts in the parallel version) containing the m^2 values

$$a_1 \mid a_1, a_2 \mid a_1, a_2, a_3 \mid \cdots \mid a_{m-1}, a_m \mid a_m,$$

where $\mid$ separates subblocks (this is just for the sake of clarity). We use a similar encoding for $\mathbf{b}$, but with subblocks in reverse order:

$$b_1 \mid b_2, b_1 \mid b_3, b_2, b_1 \mid \cdots \mid b_m, b_{m-1} \mid b_m.$$

This can be derived using $(2m-1)$ maskings and $(2m-2)$ shifts for the vector $\mathbf{a}$ if the result fits in a single ciphertext (and somewhat more masking operations otherwise). For $\mathbf{b}$, we first need to reverse the order of the characters, which can be done using m maskings and m shifts, and then we proceed similar to $\mathbf{a}$. Since the expansion operation for $\mathbf{b}$ is slightly more costly, it makes sense to use the fixed input ciphertext as the second argument in all edit distance computations, because we then only have to compute the above vector once.

By computing the difference of these M ciphertext pairs, squaring and evaluating the polynomial $e(x)$, we end up with M ciphertexts containing the $\delta_{i,j}$ in the order in which they are needed. This function is called $\mathsf{PrecompDeltas}(\mathsf{ct}_a, \mathsf{ct}_b)$. Finally, we can formulate the whole homomorphic edit distance computation in Algorithm 5. This function uses the following GBFV subroutines:

- Enc: encryption.
- $\oplus, \ominus$ and $\otimes$: shorthand for $\mathsf{GBFV.Add}$, $\mathsf{GBFV.Sub}$ and $\mathsf{GBFV.Mul}$.
- $\mathsf{ShiftRight}$ and $\mathsf{ShiftLeft}$: circular shift to the right/left by one position, which requires a single automorphism.
- $\mathsf{Eval}(g, [\mathsf{ct}_1, \ldots, \mathsf{ct}_k])$: evaluates multivariate polynomial g on $[\mathsf{ct}_1, \ldots, \mathsf{ct}_k]$.
- $\mathsf{ExtractMinusDelta}(C, k)$: extracts $-\delta_{i,j}$ with $i + j = k$ from the precomputed values in C. This requires one masking and one shift when these are contained in a single ciphertext, otherwise two maskings and two shifts.

The algorithm consists of two main loops: one for the top-left and one for the bottom-right differences. We use a temporary variable ct_T that holds the partial result for ct_H in the first loop and for ct_V in the second loop. This is in order not to overwrite the previous value of ct_H (resp. ct_V), which is still used on the subsequent line.

4.2 Complexity Analysis

The complexity and number of levels now follow from the description given in Algorithm 5. Precomputing and extracting $\delta_{i,j}$ requires depth $\lceil \log_2(|\mathcal{A}|) \rceil + 3$. Each evaluation of the polynomial g requires depth 3, so the total depth for all evaluations of g is $6m - 3$. The final masking has depth 1, but it can be omitted (in that case, the other slots contain garbage). Algorithm 5 without masking on input strings of length m from an alphabet $\mathcal{A}$ thus requires depth

$$\lceil \log_2(|\mathcal{A}|) \rceil + 6m.$$

The evaluation of g is the only subroutine in the two main loops of our algorithm that accumulates multiplicative noise growth. As a consequence, it suffices to bootstrap $\mathsf{ct}_{\mathsf{min}}$ every iteration, or to bootstrap ct_H and ct_V once in L iterations, where L is defined as

Algorithm 5 Homomorphic edit distance

Require: $\mathsf{ct}_a, \mathsf{ct}_b$ that encrypt input strings **a** and **b** of length m
Ensure: $\mathsf{ct}_{\mathsf{acc}}$ that encrypts Levenshtein edit distance in its first slot

function EditDistance($\mathsf{ct}_a, \mathsf{ct}_b$)
  $\mathsf{ct}_H, \mathsf{ct}_V \leftarrow \mathsf{Enc}([1, 0, \ldots, 0])$ ▷ Length-m vectors
  $\mathsf{ct}_{\mathsf{acc}} = \mathsf{Enc}([m, 0, \ldots, 0])$ ▷ Accumulator for result
  $C = (\mathsf{ct}_1, \ldots, \mathsf{ct}_M) \leftarrow \mathsf{PrecompDeltas}(\mathsf{ct}_a, \mathsf{ct}_b)$
  for $k \leftarrow 2$ to m **do**
    $\mathsf{ct}_{-\delta} \leftarrow \mathsf{ExtractMinusDelta}(C, k)$
    $\mathsf{ct}_{\mathsf{min}} \leftarrow \mathsf{Eval}(g, [\mathsf{ct}_{-\delta}, \mathsf{ct}_H, \mathsf{ct}_V])$ ▷ Minimum using polynomial g
    $\mathsf{ct}_T \leftarrow [1, 1, \ldots, 0] \ominus \mathsf{ct}_V \oplus \mathsf{ct}_{\mathsf{min}}$ ▷ First $k-1$ entries are 1's
    $\mathsf{ct}_V \leftarrow [1, 1, \ldots, 0] \ominus \mathsf{ct}_H \oplus \mathsf{ct}_{\mathsf{min}}$ ▷ First k entries are 1's
    $\mathsf{ct}_H \leftarrow \mathsf{ShiftRight}(\mathsf{ct}_T) \oplus [1, 0, \ldots, 0]$ ▷ Set $H[1] = 1$
  end for
  for $k \leftarrow m+1$ to $2m$ **do**
    $\mathsf{ct}_{-\delta} \leftarrow \mathsf{ExtractMinusDelta}(C, k)$
    $\mathsf{ct}_{\mathsf{min}} \leftarrow \mathsf{Eval}(g, [\mathsf{ct}_{-\delta}, \mathsf{ct}_H, \mathsf{ct}_V])$ ▷ Minimum using polynomial g
    $\mathsf{ct}_T \leftarrow [1, 1, \ldots, 0] \ominus \mathsf{ct}_H \oplus \mathsf{ct}_{\mathsf{min}}$ ▷ First $2m-k+1$ entries are 1's
    $\mathsf{ct}_H \leftarrow [1, 1, \ldots, 0] \ominus \mathsf{ct}_V \oplus \mathsf{ct}_{\mathsf{min}}$ ▷ First $2m-k$ entries are 1's
    $\mathsf{ct}_V \leftarrow \mathsf{ShiftLeft}(\mathsf{ct}_T)$
    $\mathsf{ct}_{\mathsf{acc}} \leftarrow \mathsf{ct}_{\mathsf{acc}} \oplus \mathsf{ct}_T$
  end for
  return $\mathsf{ct}_{\mathsf{acc}} \otimes [1, 0, \ldots, 0]$ ▷ First entry contains actual distance
end function

the ratio of the remaining noise budget after bootstrapping, to the consumed noise budget in one evaluation of g.

In the first and last quarter of the iterations, less than half of the plaintext vector contains useful data. Therefore, if we take the strategy of bootstrapping both ct_H and ct_V, we can combine them in a single ciphertext such that only one bootstrapping per iteration is required. This combination requires extra masking operations, but those can be folded for free in the computation of g. The time complexity is summarized in Table 1.

5 Evaluation

5.1 Implementation

We implemented our bootstrapping method and the edit distance application in the SEAL FHE library [SEA23]. All experiments below were performed on a 14-core M4 Pro machine with 64 GB RAM, and in a single thread. We take the same bootstrapping parameters as the previous work [GV25]: ring dimension $n = 2^{14}$, ciphertext modulus $q \approx 2^{420}$, noise cut-off parameter $B = 15$ for digit removal, a ternary secret key distribution with Hamming

Table 1: Complexity of homomorphic edit distance computation for input length m, alphabet $\mathcal{A}$, duplication size M, and L evaluations of g between two bootstrappings

PT × CT	$9m + M\lvert\mathcal{A}\rvert$
CT × CT	$8m + 2M\sqrt{\lvert\mathcal{A}\rvert}$
Automorphisms	$9m$
Bootstrappings	$\min(3/L, 2) \cdot (m-1)$

weight $h = 256$, and $\tilde{h} = 32$ for sparse secret encapsulation [BTH22]. This results in a security level of 128 bits. The baselines were re-evaluated due to use of a different CPU.

5.1.1 Bootstrapping

Table 2 and Table 3 compare our new GBFV bootstrapping with the results from [GV25]. These GBFV parameter sets have 1024 and 4096 slots respectively, corresponding to 11 and 14 bits of multiplication noise. Due to the high noise growth of noisy expansion, the baseline algorithm decomposes the SlotToCoeff and CoeffToSlot transformations into a very limited number of only two stages. In contrast, we can decompose in more stages, while still having a comparable noise growth. For example, the four-stage decomposition of Table 2 has approximately the same remaining noise budget as the baseline, but the execution time is only 0.81 seconds instead of 1.29 seconds. On the other hand, if we decompose in two stages, our algorithm is 0.21 seconds slower than the baseline, but we have 30 bits extra remaining noise budget.

Table 4 shows experimental results for our new bootstrapping under a varying number of slots and stages. When there is only a single slot, we can bootstrap extremely fast and noisy expansion consumes almost no levels. In applications with a larger number of slots (such as encrypted edit distance), bootstrapping is slower and consumes more noise. However, amortizing the execution time and remaining noise capacity over all slots, the best performance is reached in the last column with 4096 slots.

5.1.2 Encrypted Edit Distance

Table 5 contains timings for the homomorphic edit distance application. We use the same parameter set as the third column of Table 3, which results in the best amortized performance. In particular, this parameter set has 93 remaining bits, which allows $L = 2$ evaluations of g between two bootstrappings. We use the variant of our algorithm that bootstraps both ct_H and ct_V every L iterations, which is the best strategy for $L \geq 2$. This table also compares to the TFHE-based Leuvenshtein [LDS+25] method for homomorphic edit distance computation. The numbers of Leuvenshtein are copied from their paper, where they used a dual AMD EPYC 9174F 16-core CPU. To allow a fair comparison, we use the same values of m, except for the second test because we are restricted to powers of two. We also use the same 7-bit ASCII alphabet.

The table indicates results for the parallel version of our algorithm. All benchmarks compute the edit distance for a full batch of ℓ/m strings. This setting utilizes the packing capability of GBFV to its maximum extent, and it results in $M = m$, meaning that the precomputation of $\delta_{i,j}$ can be stored in m intermediate ciphertexts. Note that BFV cannot be bootstrapped at ring dimension $n = 2^{14}$ (at 128-bit security level and for a prime with decent splitting properties), so using BFV would dramatically increase the latency.

Table 2: Comparison to the baseline [GV25] with $n = n' = 2^{14}$, $n'' = 2^{10}$ and $p = 2^{16} + 1$

Bootstrapping algorithm		Baseline	Ours	Ours	Ours
Number of stages s		2	2	3	4
Noise (bits)	Initial	317	317	317	317
	Noisy expansion	111	81	96	110
	Digit removal	82	82	82	82
	Remaining	124	154	139	125
Execution time (sec)	Noisy expansion	0.95	1.16	0.61	0.47
	Digit removal	0.34	0.34	0.34	0.34
	Total	1.29	1.50	0.95	0.81

Table 3: Comparison to the baseline [GV25] with $n = n' = 2^{14}$, $n'' = 2^{12}$ and $p = 2^{16} + 1$

Bootstrapping algorithm		Baseline	Ours	Ours	Ours
Number of stages s		2	2	3	4
Noise (bits)	Initial	317	317	317	317
	Noisy expansion	114	88	111	134
	Digit removal	113	113	113	113
	Remaining	90	116	93	70
Execution time (sec)	Noisy expansion	0.95	1.16	0.66	0.57
	Digit removal	0.35	0.35	0.35	0.35
	Total	1.30	1.51	1.01	0.92

Table 4: Results for varying number of slots with $n = 2^{14}$, $n'/n'' = 2^2$ and $p = 2^{16} + 1$

Number of slots $\ell = n''$		1	16	256	4096
Number of stages s		0	1	2	3
Noise (bits)	Initial	317	317	317	317
	Noisy expansion	38	58	78	111
	Digit removal	109	109	112	113
	Remaining	170	150	127	93
Execution time (sec)	Noisy expansion	0.14	0.35	0.53	0.66
	Digit removal	0.35	0.35	0.35	0.35
	Total	0.49	0.70	0.88	1.01

Our amortized performance is up to 79× better than Leuvenshtein. This is a result of the parallel nature of GBFV, which allows us to process multiple edit distance computations in the same ciphertext. In terms of latency, Leuvenshtein is faster than our method for a small number of $m = 8$ characters. However, we are still quite close to their performance (the slowdown is a factor of 6) thanks to the very small bootstrapping latency of GBFV. For a larger number of $m = 128$ or even $m = 256$ characters, we do beat their latency because each ciphertext holds longer (but fewer) input strings. Our bootstrapping key is somewhat larger than theirs (413 MB versus 55 MB). Most of the computation time in our algorithm is used by bootstrapping (the other operations only account for roughly 37%).

5.2 Comparison to Other Works

5.2.1 Bootstrapping GBFV with CKKS

A recent work from Kim [Kim25] investigates alternative methods to bootstrap GBFV, but in a different setting. This method can bootstrap a wider parameters range (notably including the CLPX scheme), but it comes with two disadvantages: the ring dimension n is 4 times higher, which leads to 20 times slower results, and the denoising factor is tightly coupled to the precision of CKKS bootstrapping. Therefore, Kim's method is most useful in settings of extremely high-precision arithmetic, such as the CLPX scheme (which we cannot bootstrap since we convert to regular BFV with plaintext modulus p). We believe that Kim's method will only be more efficient than ours for a plaintext modulus that is at least a few hundred bits wide.

5.2.2 RMFE-Based BGV Bootstrapping

A recent work [ALJ+25] improves BGV bootstrapping using a reverse multiplication-friendly embedding (RMFE). Compared to our bootstrapping, this is an order or magnitude slower because of the larger ring dimension, and also the number of slots in the RMFE

Table 5: Timings (sec) for edit distance and comparison to Leuvenshtein (LVS)

Algorithm	LVS	LVS	LVS	Ours	Ours	Ours
String size m	8	100	256	8	128	256
Batch size ℓ/m	–	–	–	512	32	16
Equality tests	–	–	–	5.10	82	167
Bootstrappings	–	–	–	11.1	193	387
Other operations	–	–	–	1.50	26	57
Total latency	2.83	439	2903	17.7	301	611
Amortized time	2.83	439	2903	0.036	9.41	38.2
Speedup	–	–	–	79×	–	76×

method is quite limited (typically not more than a hundred). An interesting feature of their method is that it natively supports small-characteristic fields, whereas we are restricted to large characteristics. In practice, we think that large characteristics may suffice for many FHE applications, because any computation can be interpolated over a given finite field.

5.2.3 Multiple Precision CKKS

Cheon et al. [CCKS23] proposed a Mult^2 algorithm that multiplies two CKKS ciphertexts with asymptotically twice the cost but half the modulus consumption of a normal multiplication. This is enabled via a new pair representation by breaking each ciphertext in two components. A quantitative comparison is difficult because of the approximate nature of CKKS, so instead we qualitatively compare GBFV bootstrapping to their Mult^2 algorithm.

Our improved GBFV bootstrapping is much closer to Mult^2 than the original GBFV bootstrapping: all our bootstrapping components (except for InProd) can be evaluated in the GBFV scheme, similarly to how all CKKS bootstrapping components (except for $\mathsf{ModRaise}$) can be evaluated in pair representation of ciphertexts. Similarly, both techniques reduce the modulus consumption by exploiting the underlying high precision, but they cannot reduce the contribution inherent to the ring dimension and secret key distribution.

We argue that the asymptotic latency of GBFV bootstrapping can be reduced even more than CKKS in pair representation. If $n' = 2n''$, the modulus consumption of GBFV is roughly half that of regular BFV. Note that in this case, we can scale down both the ring dimension and bit-width of the ciphertext modulus by a factor of 2, resulting in 4 times lower latency. The same trick is applicable to CKKS bootstrapping, but its latency can only be halved, because Mult^2 is roughly twice as expensive as a regular Mult. Neither work can asymptotically improve the throughput because of the lower number of slots.

Finally, we note that CKKS tuple multiplication (i.e. Mult^t instead of Mult^2) involves a quadratic number of $\mathcal{O}(t^2)$ tensorings. For large t, we therefore expect tensoring to dominate in execution time over relinearization, which will saturate the latency reduction at a certain point. This is not the case in our algorithm.

5.2.4 Low-Latency CKKS Bootstrapping

Cheon et al. [CHKS25] recently proposed a low-latency CKKS bootstrapping algorithm called SHIP, which is designed for parallel computing environments. SHIP can bootstrap in ring dimension $n = 2^{13}$ (half that of our experiments), which results in a latency of around 0.2 seconds for 4096 slots. However, this parameter set has only one remaining multiplicative level, supports limited precision (1 to 5 bits) and uses more computing resources (a 32-core machine). Concurrently, two other works [CK25, CS25] have also shown how to reduce the latency of CKKS bootstrapping with related tricks. A quantitative comparison to these works is not possible because they use CKKS encoding.

6 Conclusion

This paper improved the bootstrapping performance of GBFV to only one second. We also showed its advantage over TFHE in homomorphic edit distance computation – an application that requires moderate packing density. Interesting future work is to integrate our bootstrapping in other software libraries [OPP25, MBTPH20] and to use it in more FHE applications. Moreover, it would be interesting to apply techniques from other FHE schemes, such as mean compensation [dRDV25], to GBFV as well.

Acknowledgements

This work is supported in part by CyberSecurity Research Flanders with reference number VR20192203 and by the Research Council KU Leuven grant C14/24/099. In addition, this work is also supported in part by the European Commission through the Horizon 2020 research and innovation program under grant agreement ISOCRYPT ERC Advanced Grant 101020788. Robin Geelen is funded by Research Foundation – Flanders (FWO) under a PhD Fellowship fundamental research (project number 1162125N). Any opinions, findings and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the ERC, the European Union, CyberSecurity Research Flanders or the FWO. The authors thank Wouter Legiest for discussions about edit distance computation and Jonas Bertels for discussions about the NTT.

References

[ALJ+25] Khin Mi Mi Aung, Enhui Lim, Sim Jun Jie, Benjamin Hong Meng Tan, and Huaxiong Wang. Bootstrapping with RMFE for fully homomorphic encryption. In *PKC (5)*, volume 15678 of *Lecture Notes in Computer Science*, pages 71–101. Springer, 2025.

[AP13] Jacob Alperin-Sheriff and Chris Peikert. Practical Bootstrapping in Quasi-linear Time. In *CRYPTO (1)*, volume 8042 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2013.

[BGV14] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) Fully Homomorphic Encryption without Bootstrapping. *ACM Trans. Comput. Theory*, 6(3):13:1–13:36, 2014.

[Bra12] Zvika Brakerski. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *CRYPTO*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886. Springer, 2012.

[BTH22] Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Bootstrapping for Approximate Homomorphic Encryption with Negligible Failure-Probability by Using Sparse-Secret Encapsulation. In *ACNS*, volume 13269 of *Lecture Notes in Computer Science*, pages 521–541. Springer, 2022.

[CCKS23] Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Damien Stehlé. Homomorphic Multiple Precision Multiplication for CKKS and Reduced Modulus Consumption. In *CCS*, pages 696–710. ACM, 2023.

[CCLS19] Jung Hee Cheon, Hyeongmin Choe, Donghwan Lee, and Yongha Son. Faster Linear Transformations in HElib, Revisited. *IEEE Access*, 7:50595–50604, 2019.

[CCS19] Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved Bootstrapping for Approximate Homomorphic Encryption. In *EUROCRYPT (2)*, volume 11477 of *Lecture Notes in Computer Science*, pages 34–54. Springer, 2019.

[CDKS21] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient Homomorphic Conversion Between (Ring) LWE Ciphertexts. In *ACNS (1)*, volume 12726 of *Lecture Notes in Computer Science*, pages 460–479. Springer, 2021.

[CH18] Hao Chen and Kyoohyung Han. Homomorphic Lower Digits Removal and Improved FHE Bootstrapping. In *EUROCRYPT (1)*, volume 10820 of *Lecture Notes in Computer Science*, pages 315–337. Springer, 2018.

[CHKS25] Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. SHIP: A shallow and highly parallelizable CKKS bootstrapping algorithm. In *EUROCRYPT (3)*, volume 15603 of *Lecture Notes in Computer Science*, pages 398–428. Springer, 2025.

[CHM+25] Hyunho Cha, Intak Hwang, Seonhong Min, Jinyeong Seo, and Yongsoo Song. MatriGear: Accelerating Authenticated Matrix Triple Generation with Scalable Prime Fields via Optimized HE Packing. In *SP*, pages 2453–2471. IEEE, 2025.

[CK25] Jean-Sébastien Coron and Robin Köstler. Low-Latency Bootstrapping for CKKS using Roots of Unity. Cryptology ePrint Archive, Paper 2025/651, 2025. URL: https://eprint.iacr.org/2025/651.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *ASIACRYPT (1)*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437. Springer, 2017.

[CKL15] Jung Hee Cheon, Miran Kim, and Kristin E. Lauter. Homomorphic Computation of Edit Distance. In *Financial Cryptography Workshops*, volume 8976 of *Lecture Notes in Computer Science*, pages 194–212. Springer, 2015.

[CLPX18] Hao Chen, Kim Laine, Rachel Player, and Yuhou Xia. High-Precision Arithmetic in Homomorphic Encryption. In *CT-RSA*, volume 10808 of *Lecture Notes in Computer Science*, pages 116–136. Springer, 2018.

[CS25] Jean-Sébastien Coron and Tim Seuré. PaCo: Bootstrapping for CKKS via Partial CoeffToSlot. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 36–67. Springer, 2025.

[dRDV25] Thomas de Ruijter, Jan-Pieter D'Anvers, and Ingrid Verbauwhede. Don't be mean: Reducing Approximation Noise in TFHE through Mean Compensation. Cryptology ePrint Archive, Paper 2025/809, 2025. URL: https://eprint.iacr.org/2025/809.

[FV12] Junfeng Fan and Frederik Vercauteren. Somewhat Practical Fully Homomorphic Encryption. Cryptology ePrint Archive, Paper 2012/144, 2012. URL: https://eprint.iacr.org/2012/144.

[Gee24] Robin Geelen. Revisiting the Slot-to-Coefficient Transformation for BGV and BFV. *IACR Commun. Cryptol.*, 1(3):37, 2024.

[GV25] Robin Geelen and Frederik Vercauteren. Fully Homomorphic Encryption for Cyclotomic Prime Moduli. In *EUROCRYPT (3)*, volume 15603 of *Lecture Notes in Computer Science*, pages 366–397. Springer, 2025.

[Kim25] Jaehyung Kim. Bootstrapping GBFV with CKKS. Cryptology ePrint Archive, Paper 2025/888, 2025. URL: https://eprint.iacr.org/2025/888.

[LDS+25] Wouter Legiest, Jan-Pieter D'Anvers, Bojan Spasic, Nam-Luc Tran, and Ingrid Verbauwhede. Leuvenshtein: Efficient FHE-based Edit Distance Computation with Single Bootstrap per Cell. In *USENIX Security Symposium*, pages 8423–8440. USENIX Association, 2025.

[LY25] Kang Hoon Lee and Ji Won Yoon. Homomorphic Field Trace Revisited: Breaking the Cubic Noise Barrier. Cryptology ePrint Archive, Paper 2025/1088, 2025. URL: https://eprint.iacr.org/2025/1088.

[MBTPH20] Christian Vincent Mouchet, Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Lattigo: A multiparty homomorphic encryption library in go. In *Proceedings of the 8th Workshop on Encrypted Computing and Applied Homomorphic Cryptography*, pages 64–70, 2020.

[MHWW24a] Shihe Ma, Tairong Huang, Anyu Wang, and Xiaoyun Wang. Accelerating BGV Bootstrapping for Large p Using Null Polynomials over $\mathbb{Z}_{p^e}$. In *EUROCRYPT (2)*, volume 14652 of *Lecture Notes in Computer Science*, pages 403–432. Springer, 2024.

[MHWW24b] Shihe Ma, Tairong Huang, Anyu Wang, and Xiaoyun Wang. Faster BGV Bootstrapping for Power-of-Two Cyclotomics Through Homomorphic NTT. In *ASIACRYPT (1)*, volume 15484 of *Lecture Notes in Computer Science*, pages 143–175. Springer, 2024.

[Mye99] Gene Myers. A Fast Bit-Vector Algorithm for Approximate String Matching Based on Dynamic Programming. *J. ACM*, 46(3):395–415, 1999.

[OPP25] Hiroki Okada, Rachel Player, and Simon Pohmann. Fheanor: a new, modular FHE library for designing and optimising schemes. Cryptology ePrint Archive, Paper 2025/864, 2025. URL: https://eprint.iacr.org/2025/864.

[SAE+08] Zhan Su, Byung-Ryul Ahn, Ki-Yol Eom, Min-Koo Kang, Jin-Pyung Kim, and Moon-Kyun Kim. Plagiarism Detection Using the Levenshtein Distance and Smith-Waterman Algorithm. In *2008 3rd International Conference on Innovative Computing Information and Control*, pages 569–569, 2008.

[SEA23] Microsoft SEAL (release 4.1). https://github.com/Microsoft/SEAL, January 2023. Microsoft Research, Redmond, WA.

[WF74] Robert A. Wagner and Michael J. Fischer. The String-to-String Correction Problem. *J. ACM*, 21(1):168–173, 1974.

[WHS+25] Ruida Wang, Jincheol Ha, Xuan Shen, Xianhui Lu, Chunling Chen, Kunpeng Wang, and Jooyoung Lee. Refined TFHE Leveled Homomorphic Evaluation and Its Application. In *CCS*, pages 2309–2323. ACM, 2025.

[ZLS+19] Yandong Zheng, Rongxing Lu, Jun Shao, Yonggang Zhang, and Hui Zhu. Efficient and Privacy-Preserving Edit Distance Query Over Encrypted Genomic Data. In *WCSP*, pages 1–6. IEEE, 2019.